ECE
ÉCOLE D'INGÉNIEURS
ENGINEERING SCHOOL

ING3 Apprentis – Projet de programmation Python

Moteur d'analyse de journaux de sécurité

Ingestion, nettoyage et analyse en flux d'un fichier de logs de 10 Go

---

Rapport final

Équipe Yann Diarrassouba
Dhanoush Kessavane
Saber Boucioufe

Enseignant Hanay Joud

Date 11 novembre 2026

Table des matières

1 Objectifs et périmètre **2**

2 Organisation de l'équipe et méthode **2**

2.1 Répartition 2

2.2 Un contrat d'interface fixé avant de coder 2

2.3 Travail en équipe et qualité 3

3 Environnement et structure du dépôt **3**

4 Le jeu de données et son script de génération **4**

4.1 Format d'une ligne 4

4.2 Fonctionnement du script 4

4.3 Anomalies et scénarios injectés 5

5 Architecture du pipeline **5**

6 Mise en œuvre des notions demandées **6**

6.1 Modules 1 et 2 : lecture, routage et nettoyage 6

6.2 Module 3 : statistiques et détection de force brute 7

6.3 Module 4 : moteur de règles et audit 9

7 Tests **9**

8 Résultats sur le fichier de 10 Go **10**

8.1 Mesures 10

8.2 Nettoyage 11

8.3 Analyse 12

8.4 Mémoire 12

9 Difficultés rencontrées et décisions **13**

9.1 Le double sens du mot « générateur » 13

9.2 Le format des données 14

9.3 Compter les rejets plutôt que les ignorer 14

9.4 L'encodage du fichier `requirements.txt` 14

9.5 Un script de génération d'abord trop complexe 14

9.6 Ce qu'a révélé la première mesure sur 1 Go 14

9.7 Une consommation mémoire qui augmentait sans limite 14

9.8 Les performances 15

10 Limites et pistes d'amélioration **15**

Conclusion **16**

1 Objectifs et périmètre

Le sujet demande une application en ligne de commande capable d'ingérer, de filtrer, de transformer et d'analyser des journaux système et réseau volumineux, sans jamais charger l'intégralité des fichiers en mémoire. L'application doit être organisée en package Python, isolée dans un environnement virtuel et pilotée par un point d'entrée unique, `main.py`.

Les échanges avec l'enseignant ont précisé le périmètre :

- le jeu de données est un seul fichier texte de 10 à 15 Go, produit par un script que nous écrivons nous-mêmes ;
- ce fichier doit être varié et contenir des erreurs, des lignes à nettoyer et des lignes malformées ;
- le traitement complet est lancé pendant la soutenance ; un affichage des résultats au fil du traitement suffit pour l'aspect « temps réel » ;
- la définition d'une attaque, les seuils et le format des lignes sont laissés à notre choix, à condition de les justifier.

Notre programme lit le fichier ligne par ligne, rejette et compte les lignes invalides, normalise les autres, met à jour des statistiques, applique deux règles de sécurité (force brute et liste noire), affiche les alertes au fil de l'eau, puis produit un résumé et un audit des variables en mémoire. Sur notre fichier de 10 Go, soit près de 118 millions de logs, le traitement complet dure 18 minutes et 42 secondes, et le processus reste autour de 26 Mo de mémoire.

2 Organisation de l'équipe et méthode

2.1 Répartition

Tableau 1 : Répartition du travail

Membre	Fichiers	Notions principales
Yann Diarrassouba	`ingestion.py`, `transformation.py`	générateurs `yield` et `yield from`, dictionnaires, tuples, `filter`, `map`, `lambda`, `match-case`
Dhanoush Kessavane	`analytics.py`, `detection.py`	`set`, `frozenset` et opérations ensemblistes, `defaultdict`, boucles `while` et `for` avec `break`, `continue` et `else`
Saber Boucioufi	`rules.py`, `audit.py`, `cli.py`, `main.py`, `generate_logs.py`	`*args` et `**kwargs`, fonctions passées en argument, `globals()` et `locals()`, script de génération du jeu de données

2.2 Un contrat d'interface fixé avant de coder

Les trois parties s'enchaînent : ce que produit l'ingestion est consommé par l'analyse et par les règles. Pour développer en parallèle, nous avons d'abord fixé un contrat : le format exact d'un log normalisé, celui d'une alerte et la signature des fonctions de jonction. Ce contrat existe sous deux formes : `docs/contrat.md`, pour le lire, et `logengine/models.py`, où le log est décrit par un `TypedDict`. Ce second fichier permet à l'éditeur de signaler immédiatement une clé mal nommée, par exemple `log["source_ip"]` au lieu de `log["ip"]`, alors qu'un simple document ne l'aurait révélé qu'à l'assemblage.

Un jeu de huit logs de test aux résultats connus (`tests/fixtures.py`) a permis à chacun de tester sa partie sans attendre celle des autres.

2.3 Travail en équipe et qualité

- **Git** : dépôt privé sur GitHub, une branche par tâche, fusion par pull request après relecture par un autre membre.
- **PEP8** : vérifiée par l'outil Ruff, avec une limite de 79 caractères par ligne.
- **Typage** : toutes les fonctions sont annotées et vérifiées par Pylance.
- **Compatibilité** : le sujet exige Python 3.10 ou plus ; `pyproject.toml` configure Ruff et Pylance pour signaler toute syntaxe plus récente.

3 Environnement et structure du dépôt

Le projet s'exécute dans un environnement virtuel `venv` et n'utilise que la bibliothèque standard de Python. La commande `pip freeze` ne renvoie donc rien : `requirements.txt` ne contient qu'une ligne de commentaire, ajoutée pour le signaler. Nous le conservons car il fait partie de la procédure habituelle de reproduction : `pip install -r requirements.txt` reste valable, et le resterait si une bibliothèque était ajoutée. Tout le code est écrit en scripts `.py` : le projet ne contient aucun notebook.

```
main.py                 point d'entrée : assemble le pipeline
generate_logs.py        script de génération du jeu de données
logengine/              package principal
    models.py           types partagés (Log, Alert, Rule)
    ingestion.py        module 1 : lecture progressive
    transformation.py   module 2 : découpage, routage, nettoyage
    analytics.py        module 3 : compteurs et ensembles
    detection.py        module 3 : détection de force brute
    rules.py            module 4 : moteur de règles, liste noire
    audit.py            module 4 : introspection
    cli.py              options de la ligne de commande
scripts/measure_memory.py   mesure de la mémoire du module 3
tests/                  53 tests unitaires et jeu de test commun
data/                   échantillon de 20 lignes et liste noire
docs/                   contrat d'interface, organisation, rapport
```

Utilisation, de la démonstration sur l'échantillon fourni (option par défaut) jusqu'au fichier complet :

```
python main.py                      # démonstration sur data/sample.log
python generate_logs.py --size 10GB
python main.py --files data/generated/logs.log --threshold 3 --audit audit.json
python -m unittest
```

Les fichiers générés sont exclus du dépôt par le `.gitignore` : GitHub refuse les fichiers de plus de 100 Mo, et le jeu de données se reconstruit à l'identique avec le script.

4 Le jeu de données et son script de génération

4.1 Format d'une ligne

Chaque ligne décrit un événement par six champs séparés par `" | "` : horodatage, niveau, source, adresse IP, utilisateur et message.

```
2026-11-01T00:00:41Z | ERROR | sshd | 198.51.100.12 | user16 | login failure
2026-11-01T00:00:42Z | INFO | api | 10.0.12.8 | user3341 | {"event": "request", ...}
2026-11-01T00:00:42Z | INFO | firewall | 10.0.53.113 | - | 10.0.53.113:443/TCP
```

Le dernier champ, le message, prend trois formes : du texte brut, un objet JSON ou une connexion `ip:port/protocole`. Ce sont précisément les trois cas que le module 2 doit router avec `match-case` : chaîne, dictionnaire et tuple de connexion. Le découpage `ligne.split(" | ", 5)` limite à cinq coupures, si bien qu'un message contenant lui-même le séparateur reste entier. Le champ `source` (api, nginx, sshd, firewall, db) sert de catégorie de log pour les comptages.

Tableau 2 : Formats de fichier envisagés

Format	Avantage	Raison de l'écarter
JSON (un objet par ligne)	Lecture directe avec `json.loads`	Chaque ligne devient un dictionnaire : les cas « chaîne » et « tuple » du `match-case` auraient été artificiels ; fichier plus lourd
CSV	Compact	Un message JSON contient des virgules, qu'il faut alors échapper
Texte structuré (retenu)	Lisible, compact, proche d'un journal système qui agrège plusieurs services	Demande d'écrire un analyseur de ligne, ce qui est justement l'objet des modules 1 et 2

4.2 Fonctionnement du script

`generate_lines(seed)` est un générateur sans fin : à chaque tour, il fabrique une ligne et la fournit avec `yield`. `write_logs` écrit chaque ligne aussitôt reçue et s'arrête dès que la taille demandée est atteinte : le fichier n'est jamais présent en mémoire, quelle que soit sa taille.

- **Graine fixe** (`random.seed`) : les trois membres obtiennent exactement le même fichier, donc les mêmes résultats.
- **Horodatage** : 20 événements par seconde simulée à partir du 1er novembre 2026, ce qui garantit l'ordre chronologique.
- **Population** : 20 000 clients en plage privée et 5 000 utilisateurs. Chaque utilisateur se connecte en SSH depuis son propre poste et se trompe de mot de passe dans 2 % des cas. Les autres erreurs proviennent de 2 000 clients « instables ».
- **Sources pondérées** : api 35 %, nginx 20 %, sshd 20 %, firewall 20 %, db 5 %.
- **Taille** : un Go est compté comme 1024^3 octets, la convention de l'explorateur Windows, pour que le fichier y apparaisse bien comme 10 Go.

4.3 Anomalies et scénarios injectés

Tableau 3 : Anomalies injectées par le script

Catégorie	Taux	Exemples
Lignes à nettoyer	3 %	espaces superflus autour d'un champ, niveau abrégé ou en minuscules (`warn`, `err`), source en majuscules
Lignes à rejeter	1 %	champs manquants, date impossible (mois 13), niveau inconnu, IP invalide (`300.x.x.x`), JSON tronqué, port non numérique
Lignes vides	0,2 %	ignorées sans être comptées comme rejets

Chaque ligne cassée ne porte qu'un seul défaut, afin que son motif de rejet soit sans ambiguïté. Le script insère aussi, en moyenne toutes les 4 000 lignes, une vague de force brute déléguée à un sous-générateur avec `yield from` : de 2 à 12 échecs de connexion depuis une même IP attaquante, suivis dans 30 % des cas d'un succès, c'est-à-dire d'un compte compromis. Les vagues de deux échecs restent sous le seuil : prises isolément, elles ne déclenchent pas d'alerte. Faute de fenêtre de temps, elles peuvent toutefois prolonger une vague précédente du même couple et atteindre le seuil (section 10).

Sur les 40 IP attaquantes, 20 figurent dans la liste noire. Celle-ci contient en outre sept adresses absentes du fichier généré, dont deux issues du jeu de test de référence, de sorte que les quatre opérations ensemblistes donnent des résultats différents.

5 Architecture du pipeline

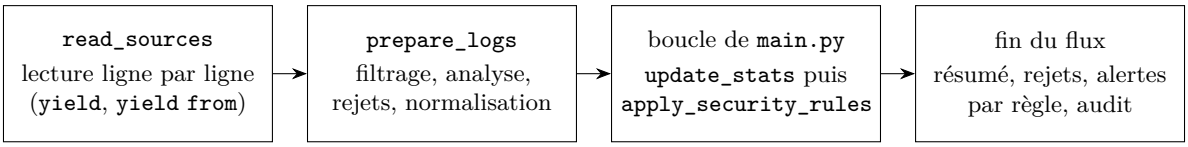


Figure 1 : Enchaînement des étapes du traitement

Les deux premières étapes ne lisent rien tant qu'on ne leur demande pas de log : elles ne font que brancher des générateurs et des itérateurs `map` et `filter` les uns sur les autres. La lecture du fichier commence quand la boucle de `main.py` demande le premier log, et chaque log est entièrement traité avant que le suivant soit lu.

Nous avons choisi une boucle unique, qui distribue chaque log à la fois aux statistiques et aux règles. Un générateur ne se parcourt qu'une fois : lancer l'analyse puis les règles dans deux boucles séparées aurait obligé à relire les 10 Go, ou à les stocker.

Les alertes sont affichées au fil du traitement, ce qui répond à l'aspect « temps réel ». Comme le fichier en produit plus de cent mille et que l'affichage dans la console est lent, seule la première alerte de chaque couple (règle, IP) est affichée, dans la limite de 20 réglable par `--max-alerts` ; toutes sont en revanche comptées dans le résumé. Une ligne de progression s'affiche tous les millions de logs.

6 Mise en œuvre des notions demandées

Tableau 4 : Correspondance entre les notions de l'énoncé et le code

Notion de l'énoncé	Emplacement	Utilisation
Générateur `yield`	`ingestion.read_lines`	fournit les lignes du fichier une à une
Sous-générateur `yield from`	`ingestion.read_sources`	enchaîne plusieurs fichiers
Dictionnaires, types variés	`transformation.to_log`	une ligne devient un dictionnaire
Tuples immuables	`to_log`, clé `connection`	métadonnées (IP, port, protocole)
`filter`, `lambda`, `filter(None, ...)`	`transformation.prepare_logs`	retire les lignes vides, puis les lignes rejetées
`map` et normalisation	`prepare_logs`, `normalize`	casse, espaces, alias de niveau, f-string
`match-case`	`to_log`, `detection._auth_status`	route texte, dictionnaire et tuple ; reconnaît une connexion
`set`	`analytics.update_stats`	IP actives et IP ayant généré des erreurs
`frozenset`, `| & - ^`	`rules.load_blacklist`, `analytics.summarize`	liste noire immuable croisée avec les IP actives
`defaultdict`	`analytics`, `main.py`	comptes par niveau, source, utilisateur, motif de rejet et règle
`while`, `for`, `break`, `continue`, `else`	`detection.detect_bruteforce`	fenêtre glissante des derniers statuts de connexion
`*args`, `**kwargs`	`rules.apply_security_rules`	moteur qui applique un nombre quelconque de règles
`globals()`, `locals()`	`main.py`, `audit.export_audit`	rapport JSON des variables en mémoire

6.1 Modules 1 et 2 : lecture, routage et nettoyage

```
def read_lines(path):
    with open(path, "r", encoding="utf-8") as fichier:
        for ligne in fichier:
            yield ligne.rstrip("\n")

def read_sources(paths):
    for path in paths:
        yield from read_lines(path)

def prepare_logs(lines, rejected):
    lignes_non_vides = filter(lambda ligne: bool(ligne.strip()), lines)
    logs_parses = map(lambda ligne: to_log(ligne, rejected), lignes_non_vides)
    logs_valides = filter(None, logs_parses)
    return map(normalize, logs_valides)
```

`read_lines` fournit une ligne puis se met en pause jusqu'à ce qu'on lui demande la suivante : à aucun moment le fichier n'est placé dans une liste. `read_sources` réutilise cette fonction pour enchaîner plusieurs fichiers dans l'ordre donné, sans dupliquer la logique de lecture. Une ligne vide est écartée par le premier `filter`, avant toute analyse : elle n'est donc pas comptée comme un rejet.

`to_log` découpe la ligne et valide successivement le nombre de champs, la date, l'IP et le niveau. Au premier défaut, elle incrémente le compteur du motif correspondant et renvoie `None`, que `filter(None, ...)` élimine. Nous avons choisi de nous arrêter à la première erreur : une ligne

qui contient trois anomalies n'est comptée qu'une fois, ce qui garde des compteurs simples à interpréter, au prix de ne pas connaître toutes les anomalies d'une même ligne. L'horodatage est validé avec `datetime`, mais conservé sous forme de chaîne au format ISO 8601 : ce format se trie et se compare directement, reste lisible dans les alertes et dans l'export JSON, et évite de conserver un objet supplémentaire par log. Un log réunit ainsi des types variés : chaînes, dictionnaire, tuple contenant un entier et `None`.

Le message est ensuite converti selon son allure. S'il commence par une accolade, `json.loads` tente de le décoder, et un JSON incomplet est rejeté. Sinon, l'expression régulière `CONNEXION_RE`, compilée une seule fois hors de la fonction, vérifie avec `fullmatch` que le message entier a la forme `ip:port/protocole`. Reconnaître la forme ne suffit pas : le port doit aussi être un entier compris entre 0 et 65 535, et l'adresse une IPv4 valide. Le log distingue ainsi deux adresses qui peuvent différer : `ip`, la source de la ligne, et `connection`, le tuple extrait du message. Le résultat est enfin routé :

```
match candidat:
    case "login failure":
        payload = {"event": "login", "status": "failure"}
    case "login success":
        payload = {"event": "login", "status": "success"}
    case dict() as donnees:
        payload = donnees
    case (str(adresse), int(port), str(protocole)):
        connection = (adresse, port, protocole)
        payload = message
    case str() as texte:
        payload = texte
```

Les motifs sont examinés dans l'ordre et seul le premier qui correspond est exécuté. Les messages `login failure` et `login success` deviennent un dictionnaire, pour que la détection lise directement le statut sans réanalyser une phrase.

`normalize` uniformise les champs qui représentent la même information sous des écritures différentes (tableau 5). Les alias de niveau sont résolus par un dictionnaire et sa méthode `get`, plutôt que par une suite de conditions. La fonction construit un nouveau log au lieu de modifier celui qu'elle reçoit, et copie le payload JSON, pour éviter tout effet de bord. Un test vérifie qu'elle est idempotente : normaliser deux fois donne le même résultat.

Tableau 5 : Exemples de normalisation

Champ	Avant	Après
Niveau	`warn`, `err`, `crit`	`WARNING`, `ERROR`, `CRITICAL`
Source	`NGINX`	`nginx`
Utilisateur	`Carol`, `-`	`carol`, `None`
Connexion	`198.51.100.7:22/tcp`	`198.51.100.7:22/TCP`

6.2 Module 3 : statistiques et détection de force brute

L'état d'analyse regroupe des compteurs `defaultdict(int)` par niveau, par source (la catégorie de log) et par utilisateur, qui évitent toute `KeyError`, et deux ensembles d'IP : les IP actives et celles qui ont produit un log `ERROR` ou `CRITICAL`. En fin de traitement, `summarize` croise les IP actives avec la liste noire, reçue sous la forme d'un `frozenset` pour qu'elle ne puisse pas être modifiée pendant l'analyse. La figure 4, en section 8.3, montre le résultat de ces quatre opérations sur le fichier de 10 Go. La liste noire contient volontairement des adresses absentes des logs : sans

elles, la différence et la différence symétrique donneraient le même résultat.

Une attaque par force brute est définie comme trois échecs de connexion consécutifs pour un même couple (IP, utilisateur). Un succès remet la série à zéro et une seule alerte est levée par série. Pour chaque couple, une file (`deque`) conserve les derniers statuts :

```
while len(history) > threshold:       # on ne garde que les derniers statuts
    history.popleft()
...
for previous in history:              # tous les statuts sont-ils des échecs ?
    if previous == "failure":
        continue
    break                             # un succès dans la fenêtre : pas d'attaque
else:                                 # aucune interruption : série complète
    state["alerted"].add(key)
    return [alerte]
```

La clause `else` de la boucle `for` n'est exécutée que si la boucle s'est terminée sans `break` : elle valide la réussite complète du cycle de détection, comme le demande l'énoncé. La boucle `while` borne la mémoire utilisée par couple. Le tableau 6 déroule cet algorithme sur les huit événements de référence.

Tableau 6 : Déroulé de la fenêtre glissante sur le jeu de test (seuil de 3 ; E : échec, S : succès)

N°	Utilisateur	Événement	Fenêtre	Résultat
1	alice	E	[E] [ ] [ ]	rien
2	bob	E	fenêtre propre à bob	rien
3	alice	E	[E] [E] [ ]	rien
4	alice	consultation de page	ignoré	rien
5	alice	E	[E] [E] [E]	**alerte**
6	alice	E	[E] [E] [E]	déjà signalé
7	alice	S	[E] [E] [S]	réarmement
8	alice	E	[E] [S] [E]	`break` sur le succès

Tableau 7 : Choix de conception de la détection

Choix	Justification
Suivi par couple (IP, utilisateur) plutôt que par IP seule	Deux personnes derrière la même IP (box, réseau d'entreprise) ne cumulent pas leurs échecs.
Détection sur le contenu `login/failure`, pas sur le niveau `ERROR`	Une erreur peut venir d'une panne de service ; seul le contenu prouve un échec de connexion.
Logs `INFO` conservés jusqu'à l'analyse	Un succès de connexion est souvent en `INFO` : sans lui, une série ne serait jamais remise à zéro.
Une seule alerte par série	Évite des milliers d'alertes identiques sur le fichier de 10 Go.
Seuil passé en paramètre (`--threshold`)	Modifiable sans toucher au code ; un seuil inférieur à 1 est refusé.

6.3 Module 4 : moteur de règles et audit

```
def apply_security_rules(*args, **kwargs):
    alerts = []
    for rule in args:
        alerts = alerts + rule(**kwargs)
    return alerts

alerts = apply_security_rules(
    detect_bruteforce, check_blacklist,
    log=log, state=state, threshold=args.threshold, blacklist=blacklist,
)
```

Les règles sont passées comme des objets, sans être appelées. Le moteur transmet toute la configuration à chacune ; une règle prend ce dont elle a besoin et ignore le reste grâce à son paramètre `**options`. Ajouter une règle ne demande donc aucune modification du moteur, et toutes les règles renvoient une liste d'alertes, vide s'il n'y a rien à signaler.

`globals()` et `locals()` décrivent la portée où ils sont appelés : ils sont donc appelés dans `main()`, et leurs résultats sont transmis au module d'audit. Pour chaque variable, celui-ci exporte en JSON son type, sa taille (`sys.getsizeof`, qui mesure l'objet lui-même mais pas ce qu'il contient) et un aperçu limité à 80 caractères. L'aperçu est obtenu par `str()`, qui ne parcourt jamais un générateur : l'audit ne relance pas la lecture des données.

7 Tests

Le projet compte 53 tests unitaires (`python -m unittest`), tous réussis. Ils s'appuient sur deux jeux de référence : huit logs normalisés dont les résultats sont connus à l'avance (alerte de force brute au cinquième, alerte de liste noire au deuxième), et l'échantillon `data/sample.log`, dont les 20 lignes doivent donner 12 logs valides, une ligne vide ignorée et 7 rejets répartis par motif. Chaque test repart d'un état neuf, pour qu'aucun n'influence les autres.

Tableau 8 : Répartition des tests

Fichier	Tests	Ce qui est vérifié
`test_transformation.py`	13	routage des trois formes, rejets par motif, alias de niveau, normalisation idempotente, flux qui reste paresseux, enchaînement de plusieurs sources
`test_detection.py`	15	scénario de référence ; 0, X−1, X et X+1 échecs ; succès au milieu d'une série ; réarmement ; même IP avec deux utilisateurs ; fenêtre bornée après 100 échecs ; seuil invalide
`test_analytics.py`	7	compteurs, ensembles d'IP, opérations ensemblistes, cas vides
`test_rules.py`	4	lecture de la liste noire, règle de liste noire, moteur de règles
`test_generate_logs.py`	6	reproductibilité par la graine, conversion des tailles, provenance des erreurs, poste fixe de chaque utilisateur
`test_cli.py`, `test_audit.py`, `test_fixtures.py`	8	options et valeurs refusées, audit sans consommation d'un générateur, cohérence du jeu de référence

8 Résultats sur le fichier de 10 Go

8.1 Mesures

Les mesures ont été réalisées sur un PC fixe (Ryzen 7 9800X3D, 32 Go de RAM, SSD NVMe) avec Python 3.14, le code restant compatible avec Python 3.10 comme l'exige le sujet. Le listing ci-dessous reproduit le début et la fin de la sortie du traitement.

```
(.venv) PS C:\dev\ece\projet-logs> python generate_logs.py --size 10GB
10.00 Go écrits dans data/generated/logs.log en 230 s
(.venv) PS C:\dev\ece\projet-logs> python main.py --files data/generated/logs.log --audit audit.json
ALERTE [blacklist] 2026-11-01T00:00:41Z : activité d'une IP en liste noire : 198.51.100.12
ALERTE [bruteforce] 2026-11-01T00:00:41Z : 3 échecs de connexion consécutifs pour user16 depuis 198.51.100.12
ALERTE [blacklist] 2026-11-01T00:01:36Z : activité d'une IP en liste noire : 198.51.100.14
ALERTE [bruteforce] 2026-11-01T00:01:36Z : 3 échecs de connexion consécutifs pour user1 depuis 198.51.100.14
ALERTE [bruteforce] 2026-11-01T00:02:10Z : 3 échecs de connexion consécutifs pour user20 depuis 198.51.100.32
[...]
... 1000000 logs traités en 10 s
... 2000000 logs traités en 19 s
[...]
... 117000000 logs traités en 1113 s

=== Résumé ===
total : 117946193
levels : {'INFO': 96124429, 'WARNING': 12478113, 'ERROR': 9034138, 'CRITICAL': 309513}
users : 5002 différents
active_ips : 20040 (ex. ['10.0.44.64', '10.0.44.207', '10.0.48.83'])
error_ips : 5040 (ex. ['10.0.1.228', '10.0.1.33', '10.0.6.211'])
union : 20047 (ex. ['10.0.44.64', '10.0.44.207', '10.0.48.83'])
intersection : 20 (ex. ['198.51.100.19', '198.51.100.14', '198.51.100.22'])
difference : 20020 (ex. ['10.0.44.64', '10.0.44.207', '10.0.48.83'])
symmetric_difference : 20027 (ex. ['10.0.44.64', '10.0.44.207', '10.0.48.83'])
Lignes rejetées : 1192337
  champs_manquants : 345694
  [...]
Alertes :
  blacklist : 109260
  bruteforce : 10324
Durée totale : 1122 s
Audit exporté dans audit.json : 18 variables globales, 21 locales
```

Tableau 9 : Génération et traitement du fichier de 10 Go

Mesure	Valeur
Taille du fichier généré	10,00 Go
Durée de génération	3 min 50 s
Lignes analysées (hors lignes vides)	119 138 530
Logs valides traités	117 946 193
Durée du traitement complet	18 min 42 s
Débit moyen	environ 105 000 logs/s
Mémoire du processus (gestionnaire des tâches)	environ 26 Mo
Variables exportées par l'audit	18 globales, 21 locales

La progression affichée tous les millions de logs est régulière, à raison d'environ 9,5 secondes par million du début à la fin du fichier (figure 2) : le débit ne se dégrade pas avec le volume déjà traité.

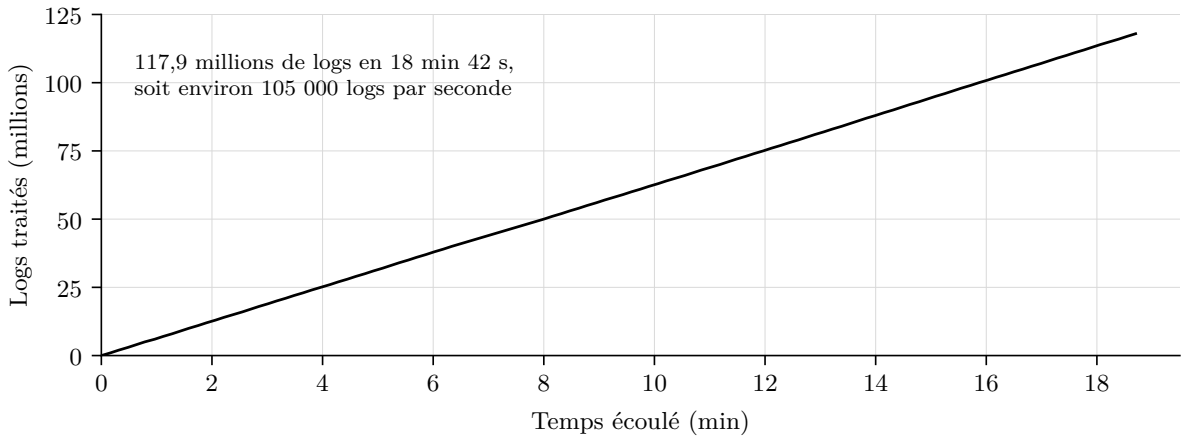


Figure 2 : Progression du traitement du fichier de 10 Go, relevée tous les millions de logs

8.2 Nettoyage

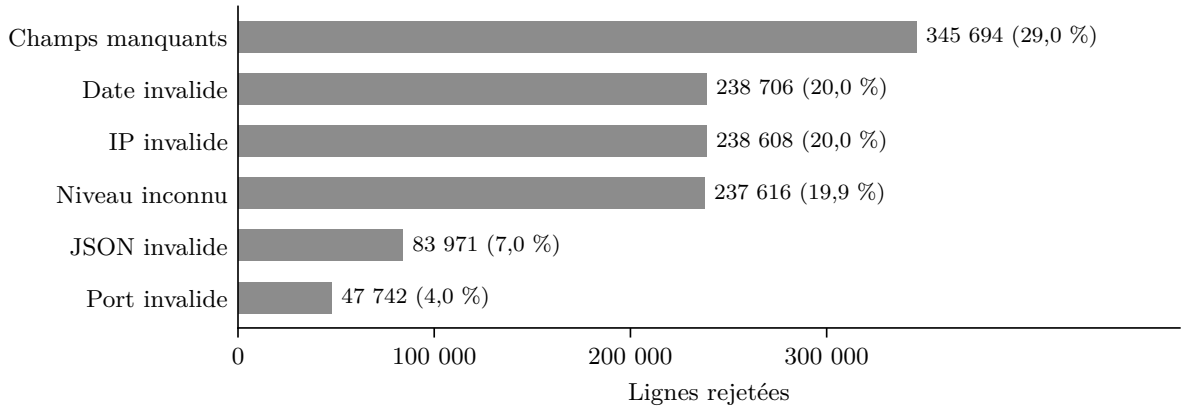


Figure 3 : Lignes rejetées par motif sur le fichier de 10 Go

Les 1 192 337 lignes rejetées représentent 1,00 % des lignes analysées, soit exactement le taux injecté par le script. Leur répartition s'explique aussi par sa conception : chacun des cinq types de défaut est tiré avec la même probabilité, d'où environ 20 % pour la date, l'IP et le niveau. Le cinquième type, un contenu invalide, se traduit selon la source par un JSON tronqué sur une ligne d'API (35 % des lignes), un port non numérique sur une ligne de pare-feu (20 %) ou un enregistrement illisible ailleurs ; ces derniers s'ajoutent aux lignes tronquées dans le motif « champs manquants ». Retrouver ces proportions confirme que chaque défaut est détecté par le bon contrôle.

8.3 Analyse

Tableau 10 : Résultats de l'analyse

Indicateur	Résultat
Logs valides	117 946 193
dont `INFO` / `WARNING`	96 124 429 / 12 478 113
dont `ERROR` / `CRITICAL`	9 034 138 / 309 513
Utilisateurs distincts	5 002
IP actives	20 040
IP ayant généré des erreurs	5 040
Alertes de force brute	10 324
Alertes de liste noire	109 260

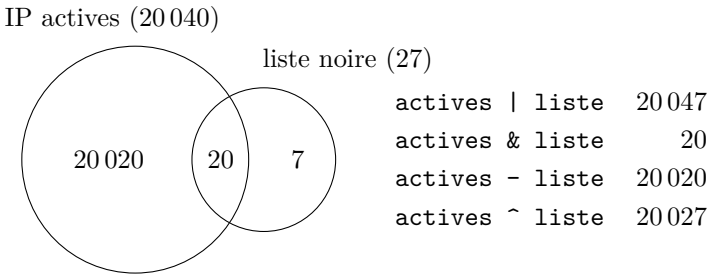


Figure 4 : Opérations ensemblistes entre les IP actives et la liste noire sur le fichier de 10 Go (surfaces non proportionnelles)

Les 20 040 IP actives correspondent aux 20 000 clients et aux 40 attaquants. Les 5 040 IP en erreur sont les postes des 5 000 utilisateurs, dont chacun finit par se tromper de mot de passe au moins une fois, et les 40 attaquants. Les 20 IP de l'intersection sont les attaquants à la fois actifs et inscrits sur la liste noire ; les 20 autres attaquants n'y figurent pas, et seule la règle de force brute permet de les repérer, ce qui montre l'intérêt de combiner les deux règles. La différence symétrique dépasse la différence de 7 : ce sont les adresses de la liste noire jamais vues dans le fichier.

Le script injecte environ 30 000 vagues d'attaque, mais n'en signale que 10 324. Les 40 attaquants visent en effet 52 comptes, soit au plus 2 080 couples (IP, utilisateur), et un couple déjà signalé ne déclenche une nouvelle alerte qu'après un succès qui réarme la détection. Les 109 260 alertes de liste noire correspondent, elles, à chacun des événements émis par les 20 attaquants inscrits.

8.4 Mémoire

Pendant tout le traitement des 10 Go, le processus est resté autour de 26 Mo : les lignes sont lues, traitées puis oubliées une à une. Le traitement n'est pas pour autant à mémoire constante : les ensembles d'IP, les compteurs par utilisateur et l'historique de connexion par couple (IP, utilisateur) grandissent avec le nombre de valeurs distinctes. Ici, ce nombre est borné : environ 20 000 IP, 5 000 utilisateurs et quelques milliers de couples suivis. La section 9 montre ce qui se passe quand il ne l'est pas.

Pour isoler ces deux effets, nous avons mesuré le module d'analyse seul avec `tracemalloc`, qui relève le pic de mémoire allouée par Python. Le script `scripts/measure_memory.py` fabrique

des logs simulés un par un avec un générateur, à graine fixe, et les transmet à `update_stats` et `detect_bruteforce`. Une première série fait varier le nombre de logs à 50 IP fixes, une seconde le nombre d'IP distinctes (tableau 11 et figure 5).

Tableau 11 : Pic mémoire du module 3 mesuré avec `tracemalloc`

Scénario	Logs	IP distinctes	Couples	Pic mémoire
Peu d'IP	10 000	50	1 000	1,11 Mo
Peu d'IP	100 000	50	1 000	1,59 Mo
Peu d'IP	1 000 000	50	1 000	1,58 Mo
Peu d'IP	100 000 000	50	1 000	1,67 Mo
Beaucoup d'IP	100 000	63 274	84 932	86,53 Mo
Beaucoup d'IP	1 000 000	631 777	850 018	902,66 Mo

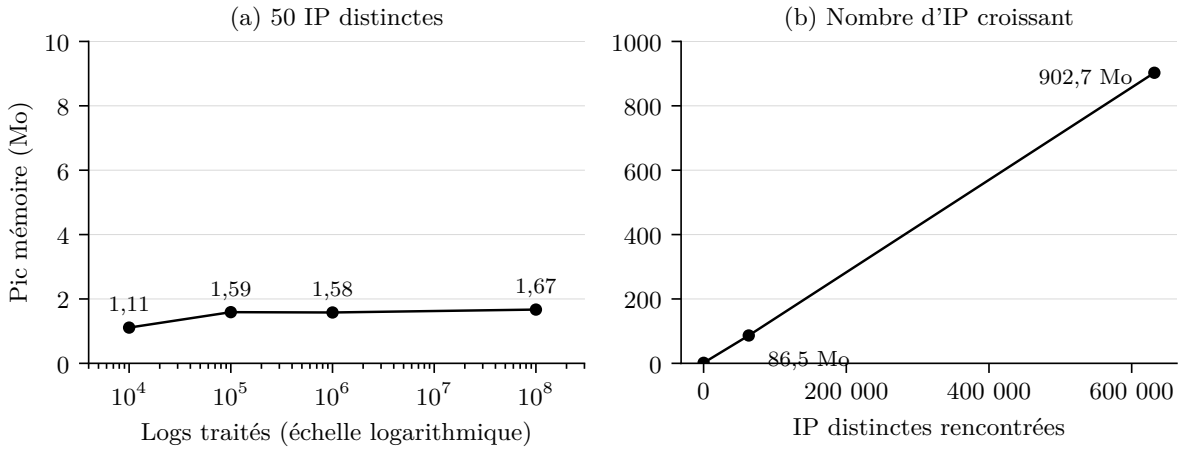


Figure 5 : Pic mémoire du module 3 selon le nombre de logs (a) et selon le nombre d'IP distinctes (b)

Passer de 10 000 à 100 millions de logs ne change presque rien : le test à 100 millions, de l'ordre de grandeur de notre fichier, a duré environ 52 minutes, et le pic est resté à 1,67 Mo du début à la fin. En revanche, dix fois plus d'IP distinctes donnent environ dix fois plus de mémoire. La mémoire dépend donc du nombre de valeurs distinctes, et non de la taille du fichier.

9 Difficultés rencontrées et décisions

9.1 Le double sens du mot « générateur »

Le module 1 de l'énoncé s'intitule « Générateur de logs » mais décrit un générateur Python qui lit un fichier, alors que la consigne complémentaire demandait un « générateur » produisant les données. Nous avons d'abord confondu les deux, avant de faire confirmer par l'enseignant qu'il fallait écrire un script de génération distinct. Nous parlons depuis de « script de génération » pour lever l'ambiguïté.

9.2 Le format des données

Notre première idée était de répartir les données sur trois fichiers (texte, JSON Lines et CSV) afin que chacun fournisse l'une des trois formes du `match-case`. La confirmation qu'un fichier unique était attendu nous a conduits au format actuel, où c'est le message de chaque ligne qui porte ces trois formes.

9.3 Compter les rejets plutôt que les ignorer

Une ligne invalide ne doit jamais interrompre le programme. Plutôt que de l'écarter silencieusement, nous comptons chaque rejet par motif dans un `defaultdict`, ce qui rend le nettoyage mesurable et vérifiable.

9.4 L'encodage du fichier `requirements.txt`

Sous Windows PowerShell, la redirection `pip freeze > requirements.txt` produit un fichier encodé en UTF-16, que Git traite comme un fichier binaire. Nous utilisons `pip freeze | Set-Content requirements.txt`.

9.5 Un script de génération d'abord trop complexe

La première version du script faisait avancer le temps par des écarts aléatoires, à l'aide d'une classe dédiée. Le résultat était plus réaliste, mais difficile à maintenir et à expliquer : nous l'avons remplacée par un horodatage calculé à partir du numéro de ligne, et nous avons simplifié le code pour rester au plus près des notions vues en cours.

9.6 Ce qu'a révélé la première mesure sur 1 Go

- **Des alertes répétées** : la règle de liste noire alerte à chaque événement d'une IP bannie, et l'affichage en direct montrait treize fois la même attaque. Nous n'affichons plus que la première alerte de chaque couple (règle, IP), le décompte restant complet.
- **Un ensemble d'IP en erreur inutilisable** : les erreurs étant réparties au hasard, chacune des 20 040 IP finissait par en produire une, et l'ensemble ne distinguait plus rien. Nous avons concentré les erreurs sur un sous-ensemble de clients « instables ».
- **Un résumé illisible** : il affichait des ensembles complets de 20 000 adresses. Il affiche désormais leur taille et quelques exemples.

9.7 Une consommation mémoire qui augmentait sans limite

Sur un fichier de test de 300 Mo, le traitement occupait 685 Mo de RAM et conservait 687 317 historiques de connexion. Extrapolé à 10 Go, cela représentait une vingtaine de gigaoctets. La cause : le script associait au hasard 20 000 IP et 5 000 utilisateurs, soit 100 millions de couples possibles, et la détection conserve un historique pour chaque couple rencontré. Les lignes, elles, étaient bien lues en flux : c'est l'état d'analyse qui grossissait. Le premier lancement sur le fichier de 10 Go l'a confirmé : le processus occupait environ 4 Go de RAM après 21 millions de logs, et la consommation continuait de monter. Nous l'avons interrompu. C'est ce que prévoyait la mesure isolée de la section 8 : 850 000 couples suffisent à dépasser 900 Mo.

Des données réalistes résolvent le problème : un utilisateur se connecte depuis son propre poste, pas depuis une machine tirée au hasard. Une fois ce comportement modélisé, le même fichier de

300 Mo n'occupe plus que 31 Mo pour 5 703 historiques, avec exactement les mêmes alertes, et le fichier complet de 10 Go reste autour de 26 Mo. Nous avons aussi abaissé le taux d'échec normal de 10 à 2 %, pour qu'un utilisateur légitime ne déclenche que rarement trois échecs consécutifs. Cette expérience montre que lire en flux ne suffit pas : il faut aussi surveiller ce que l'on accumule pendant la lecture.

9.8 Les performances

Le débit d'environ 105 000 logs par seconde permet de traiter les 10 Go en moins de vingt minutes, pendant la soutenance. Le profileur de la bibliothèque standard (`python -m cProfile`) montre que la validation de la date avec `datetime.strptime`, appelée sur chaque ligne, représente à elle seule près d'un tiers du temps de traitement, et la validation des adresses par `IPv4Address` environ un cinquième.

10 Limites et pistes d'amélioration

- **Compte compromis** : un succès qui suit une série d'échecs ne fait que réarmer la détection, alors qu'il signale probablement un mot de passe trouvé. Notre script en injecte dans 30 % des attaques ; une alerte « connexion réussie après une série d'échecs » serait la prochaine règle à ajouter.
- **Fenêtre de temps** : la règle de force brute raisonne par couple (IP, utilisateur) sans tenir compte du temps : trois échecs espacés de plusieurs jours déclenchent aussi une alerte. Sur un fichier de 300 Mo, 2 des 691 alertes visent ainsi des utilisateurs légitimes ; à l'inverse, un attaquant qui ne réussit jamais n'est signalé qu'une fois sur toute la durée simulée. Le script de génération connaissant la vérité terrain, mesurer systématiquement la précision de la détection serait l'étape suivante. Elle ne détecte pas non plus une attaque par pulvérisation, où une IP essaie un mot de passe sur de nombreux comptes ; une règle supplémentaire par IP s'ajouterait sans modifier le moteur.
- **Rejets** : une ligne n'est comptée qu'à son premier défaut ; un mode d'audit plus détaillé pourrait relever toutes les anomalies d'une même ligne.
- **Mémoire** : les structures d'état grandissent avec le nombre de valeurs distinctes. Supprimer l'historique d'un couple dès un succès ne ferait rien perdre, puisqu'un succès remet de toute façon la série à zéro : sur le fichier de 300 Mo, cette modification de deux lignes donne exactement les mêmes 691 alertes avec 582 historiques au lieu de 5 703. Les couples dont le dernier événement est un échec resteraient en mémoire ; une fenêtre de temps permettrait de les oublier aussi.
- **Robustesse** : un octet non UTF-8 dans le fichier provoque une erreur de décodage qui interrompt la lecture ; ouvrir le fichier avec `errors="replace"` en ferait un simple rejet. De même, un Ctrl+C ou une règle qui lève une exception interrompent le traitement sans résumé.
- **Liste noire** : une alerte par événement est exhaustive mais bruyante ; une alerte par IP et par période serait plus exploitable.
- **Performances** : remplacer `strptime` par `datetime.fromisoformat`, précédée d'un contrôle de la longueur et des caractères `T` et `Z`, divise par deux la durée du traitement sur un fichier de test de 150 Mo, à résultats identiques. Répartir le fichier en blocs traités par plusieurs processus irait plus loin, mais demanderait de gérer les séries d'échecs à cheval sur deux blocs.

- **Temps réel** : le programme traite un fichier existant ; suivre un fichier qui se remplit en continu, à la manière de `tail -f`, serait l'étape suivante.

Conclusion

Le projet répond aux exigences de l'énoncé : traitement en flux d'un fichier de 10 Go, nettoyage chiffré, analyse ensembliste, détection d'attaques, moteur de règles extensible et audit de l'exécution, dans un package testé et conforme à la PEP8. Fixer un contrat d'interface dès le départ nous a permis de développer les trois parties en parallèle et de les assembler sans modification. Les mesures sur de vrais volumes, enfin, nous ont fait corriger des défauts qu'aucun petit jeu de test n'aurait révélés, à commencer par la consommation de mémoire.